

Linear Regression

EECS 492

Introduction to Artificial Intelligence

Reminders

- This week's discussion on decision trees and linear regression
- HW1 deadline is 10/6

Today's Class

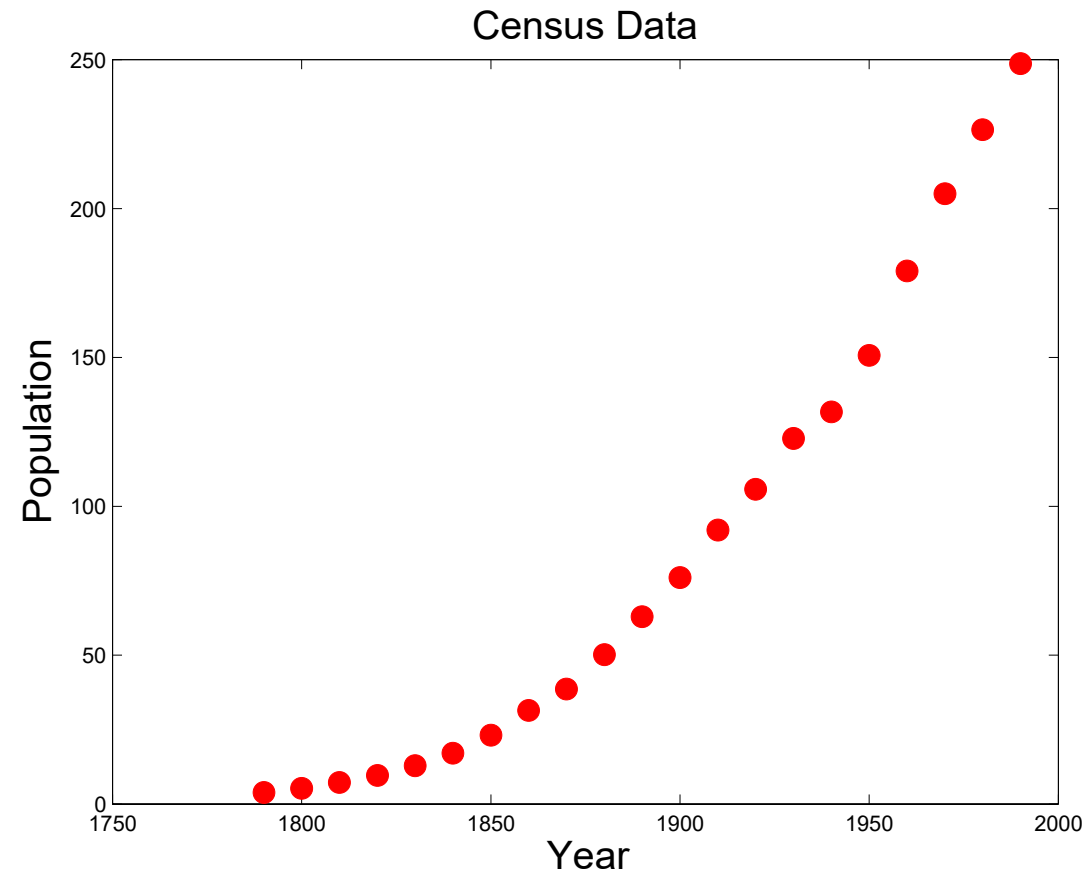
- **Univariate Linear Regression**
- **Multivariate Linear Regression**
- **Regularization**

Linear Regression

Classification and Regression

- We want to learn functions of the form:
 - $y = f(x)$
- The types of values that y takes affects the modeling that we use
 - Discrete y – classification
 - **Continuous y – regression**
- The values of x can be continuous or discrete

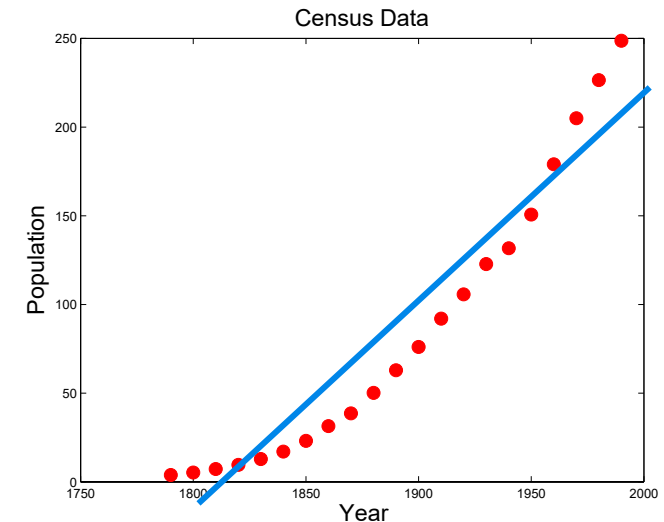
Modeling Population Changes



Linear Regression

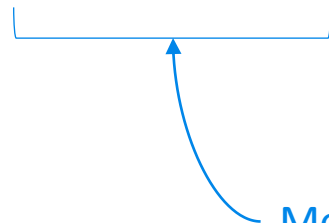
How do we think about this goal?

- We may think we want a model that **describes a fixed relationship**
 - But, this won't allow for errors! Which is unrealistic in real-world data
- Alternatively, we would like to find a model that **assumes natural variation in data**
 - The model should capture the overall pattern and allow for **random fluctuations**
- Simple linear regression!
 - A straight-line relationship **plus some noise**

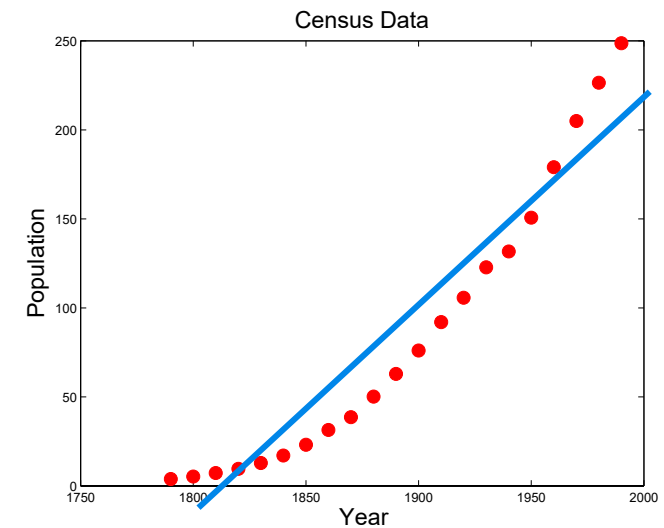


Linear Regression Assumptions

- Mean value of y for a given value of x graphs as a straight line
- Points deviate about this line of means by a random (positive or negative) amount equal to ϵ
- Formulation: $y = w_0 + w_1x + \epsilon$ ← Random error



Mean value of y for a given x



An aside: what are we assuming?

- Assumption 1:
 - Mean of probability distribution of ϵ is 0
 - Another way: average of errors over infinitely long series of experiments is 0
- Assumption 2:
 - Variance of probability distribution of ϵ is constant for all settings of independent variable x
 - For straight-line model: variance of ϵ is equal to a constant, σ^2 , for all values of x
- Assumption 3:
 - Probability distribution of ϵ is normal (Gaussian)
- Assumption 4:
 - Errors associated with any two different observations are independent
 - Error of one value of y has no effect on other y 's

An aside: what are we assuming?

- Assumption 1:
 - Mean of probability distribution of ϵ is 0
 - Another way: average of errors over infinite observations is 0
- Assumption 2:
 - Variance of ϵ is constant
 - For straight line, variance of ϵ is constant for all values of x
- Assumption 3:
 - Probability distribution of ϵ is normal
- Assumption 4:
 - Errors associated with any two different observations are independent
 - Error of one value of y has no effect on other y 's

In practice – assumptions won't hold exactly!
Usually, this still results in reliable models.

Notation:

- x : input feature
- y : output value
- w_0, w_1 : weights
- h_w : prediction function

Univariate Linear Regression

- Let's look at linear regression on one variable (**univariate**)
 - We previously defined: $y = f(x)$
 - Let $f(x)$ be a linear function of the form: $y = Ax + b$
 - We will think of the coefficients as **weights**. We can rewrite this with weights w_1 and w_0 :

$$h_w(x) = w_1x + w_0$$

- Our goal is to find the h_w that best fits these data
 - We will find the values of the weights $[w_0, w_1]$
 - Training goal: minimize the empirical loss

$$w^* = \operatorname{argmin}_w \operatorname{Loss}(h_w)$$

Empirical Loss

- L_2 : squared-error loss function
- y_j : true output value for data point j
- x_j : input feature for data point j
- $h_w(x_j)$: predicted value for data point j

- It is traditional to use the squared-error loss function L_2

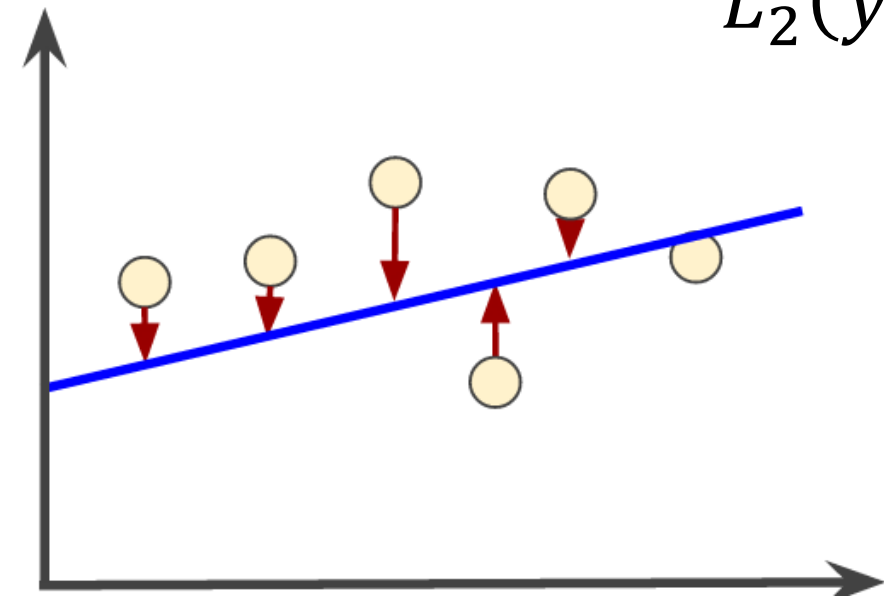
$$L_2(a, b) = (a - b)^2$$

- For one data point, we want the difference between our predicted value and the ground truth value:

$$L_2(y_i, h_w(x_i)) = (y_i - h_w(x_i))^2$$

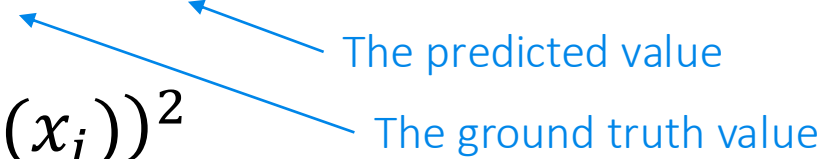
The predicted value

The ground truth value



Empirical Loss for Univariate Linear Regression

- We want to sum up the L_2 loss over all n training examples:

$$\begin{aligned} \text{Loss}(h_w) &= \sum_{i=1}^n L_2(y_i, h_w(x_i)) \\ &= \sum_{i=1}^n (y_i - h_w(x_i))^2 \\ &= \sum_{i=1}^n (y_i - (w_1 x_i + w_0))^2 \end{aligned}$$


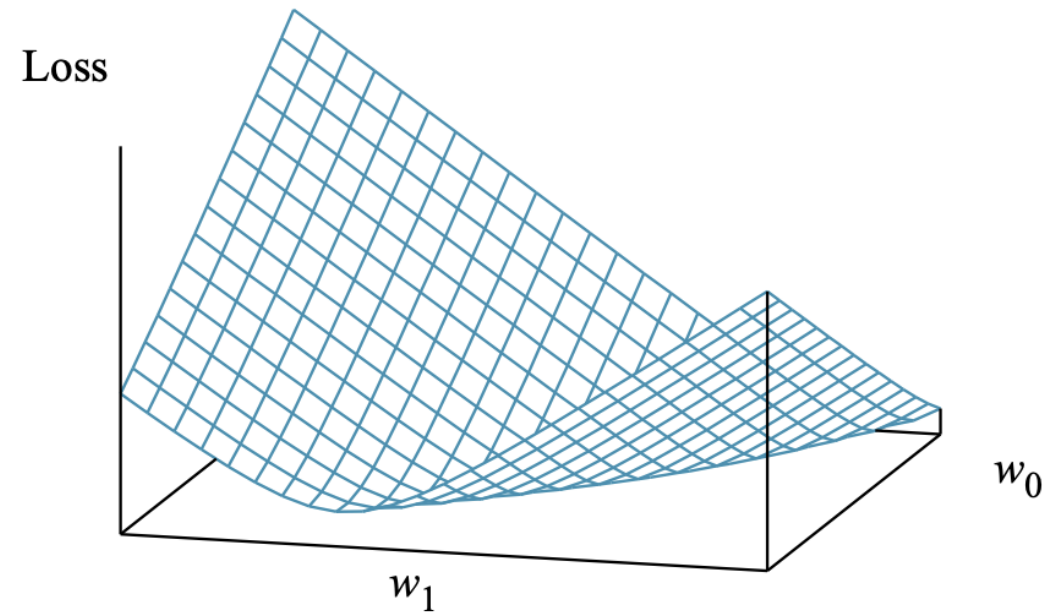
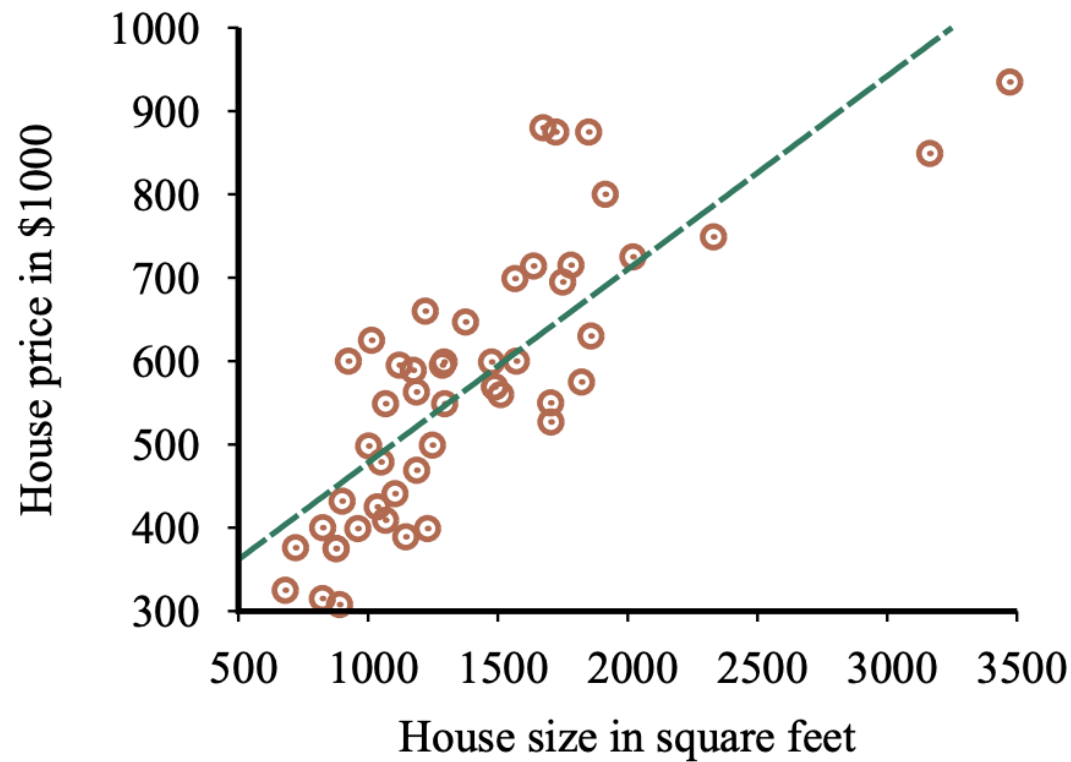
The predicted value

The ground truth value

- L_2 : squared-error loss function
- n : number of training examples
- h_w : prediction function
- y_i : true output value for data point j
- x_i : input feature for data point j
- $h_w(x_i)$: predicted value for data point j
- w_0, w_1 : weights

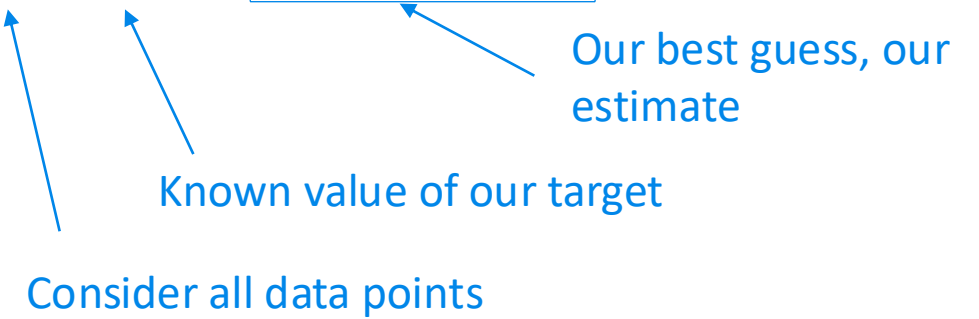
L2 loss also known as sum of squared errors

L2 loss guarantees convexity, i.e., no local minima



Note – the loss function is convex!

How do we solve this?

- Sum of squared errors: $Loss(h_w) = \sum_{i=1}^n [y_i - (w_0 + w_1 x_i)]^2$ 

Our best guess, our estimate

Known value of our target

Consider all data points

How do we solve this?

- Sum of squared errors: $Loss(h_w) = \sum_{i=1}^n [y_i - (w_0 + w_1 x_i)]^2$
- Compute partial derivatives:
 - $\frac{\delta Loss}{\delta w_0} = \sum_{i=1}^n 2[y_i - (w_0 + w_1 x_i)](-1)$
 - $\frac{\delta Loss}{\delta w_1} = \sum_{i=1}^n 2[y_i - (w_0 + w_1 x_i)](-x_i)$
- Set partial derivatives equal to 0 and simplify ...

How do we solve this?

- Sum of squared errors: $Loss(h_w) = \sum_{i=1}^n [y_i - (w_0 + w_1 x_i)]^2$

- Compute partial derivatives:

- $\frac{\delta Loss}{\delta w_0} = \sum_{i=1}^n 2[y_i - (w_0 + w_1 x_i)](-1)$

- $\frac{\delta Loss}{\delta w_1} = \sum_{i=1}^n 2[y_i - (w_0 + w_1 x_i)](-x_i)$

- Set partial derivatives equal to 0 and simplify ...

$$w_1 = \frac{n(\sum x_i y_i) - (\sum x_i)(\sum y_i)}{n(\sum x_i^2) - (\sum x_i)^2}$$

$$w_0 = \frac{\sum y_i - w_1 \sum x_i}{n}$$

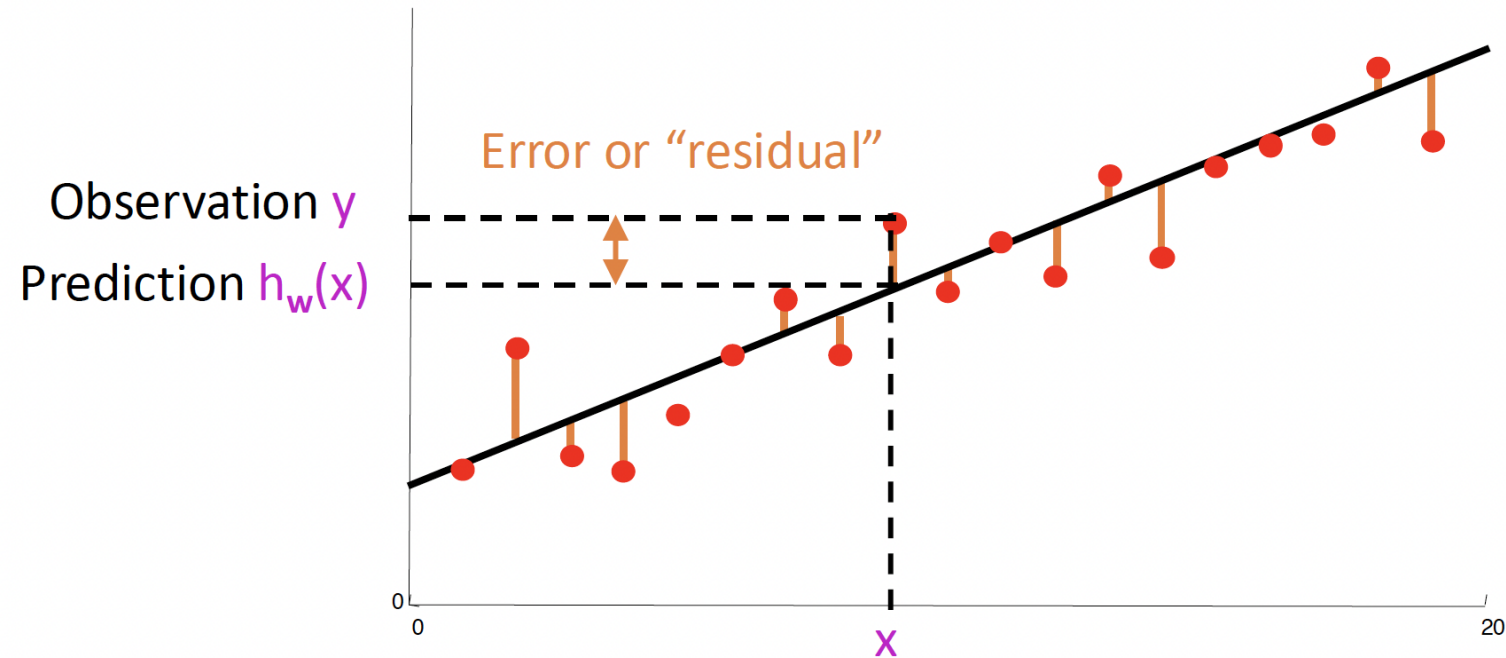
This is called the **analytical solution**
for univariate linear regression

Linear Regression Example

$$h_w(x) = w_1x + w_0$$
$$\text{Loss } h_w(x) = \sum_{j=1}^N (y_j - \hat{y}_j)^2$$

x	y
3	8
9	6
5	4
3	2

First step: calculate w_1 and w_0 that minimize the loss



Linear Regression Example

$$h_w(x) = w_1x + w_0$$
$$\text{Loss } h_w(x) = \sum_{j=1}^N (y_j - \hat{y}_j)^2$$

x	y
3	8
9	6
5	4
3	2

First step: calculate w1 and w0 that minimize the loss

✓
0s



```
import sklearn
from sklearn.linear_model import LinearRegression

X = np.asarray([3,9,5,3]).reshape(-1, 1)
y = np.asarray([8,6,4,2]).reshape(-1, 1)
reg = LinearRegression().fit(X,y)
print('slope:', reg.coef_[0])
print('intercept:', reg.intercept_)
```



```
slope: [0.16666667]
intercept:
```

Multivariate Linear Regression

- h_w : prediction function
- x_i : vector of input features for data point i
- w_0 : intercept weight
- w_j : weight for feature j
- k : number of features
- $x_{i,j}$: input feature j for data point i
- w : vector of weights

- What about more than one feature?
 - Let's say we have k features:

$$h_w(x_i) = w_0 + \sum_{j=1}^k w_j x_{i,j}$$

- We can rewrite this to make it simpler!
 - Treat the intercept, w_0 , as interacting with a dummy variable ($x_{i,0}$) which always equals 1:

$$h_w(x_i) = \sum_{j=0}^k w_j x_{i,j}$$

- Simplify using vector notation:

$$h_w(x_i) = \sum_{j=0}^k w_j x_{i,j} = \mathbf{w}^T \mathbf{x}_i$$

- h_w : prediction function
- w^* : optimal weights
- X : matrix of input features
- y : vector of output values

Finding Optimal Weights

- Again, we want weights that minimize empirical loss:

$$w^* = \operatorname{argmin}_w \operatorname{Loss}(h_w)$$

- Option #1: Solve it analytically

$$w^* = (X^T X)^{-1} X^T y$$

- X is a matrix of all the input features (each row is all the features for a single data point)
- y is a vector of true output values for each data point

$$X = \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,k} \\ \vdots & \vdots & \dots & \vdots \\ 1 & x_{n,1} & \dots & x_{n,k} \end{bmatrix}$$

$$y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

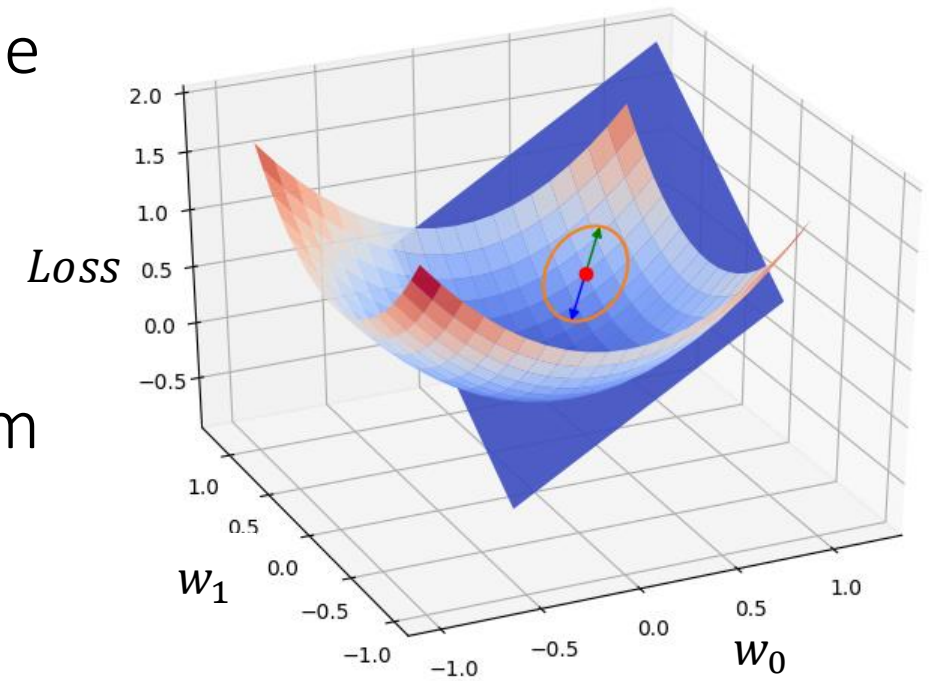
Finding Optimal Weights

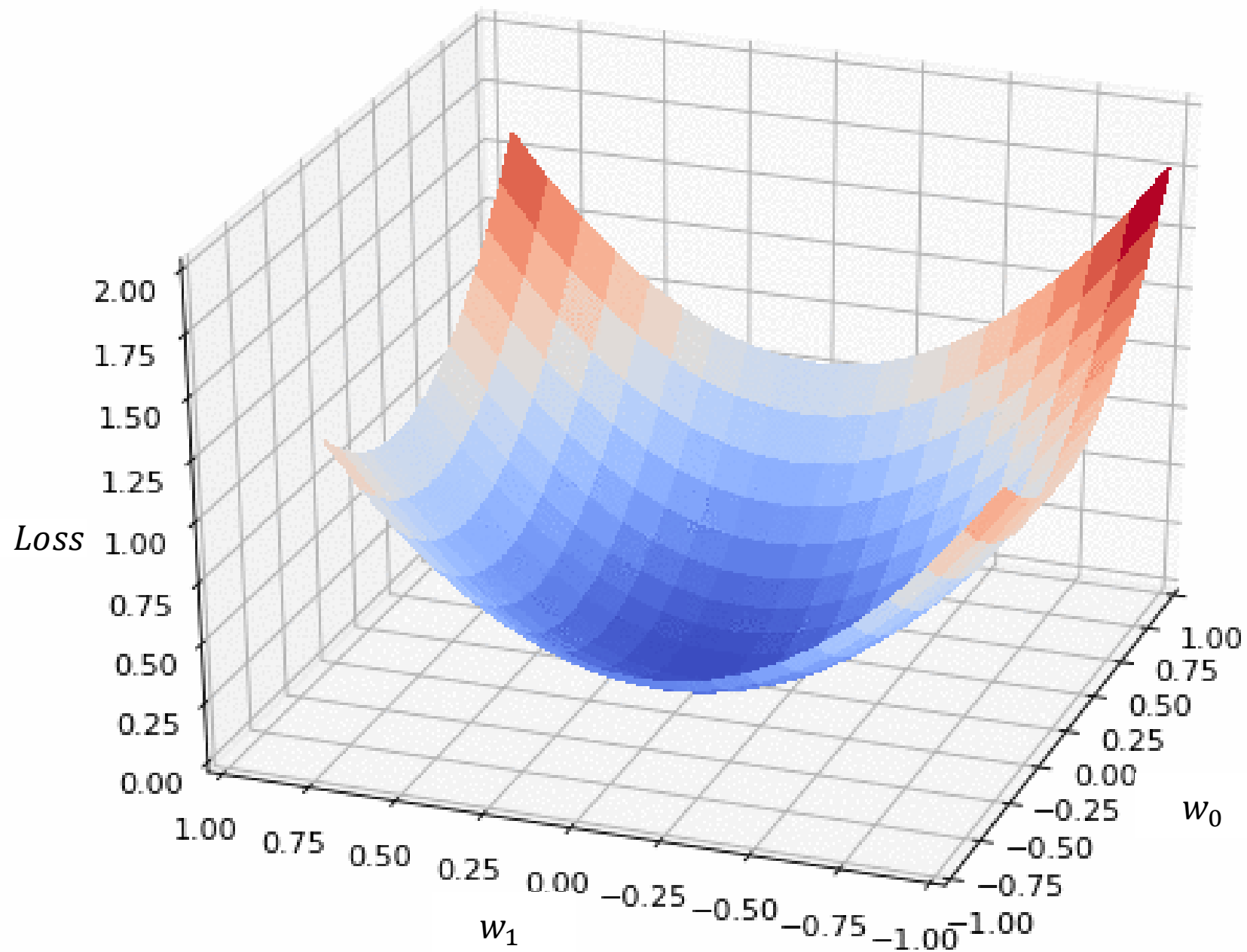
- Option #2: Gradient descent
- Minimize loss function by searching through continuous weight space and incrementally modifying parameters

- $\mathbf{w}$: vector of weights
- w_j : weight for feature j
- α : learning rate

Gradient Descent Algorithm

- Pick a random starting point $\mathbf{w}$ (a random vector of weights)
- For each weight, estimate the gradient $\frac{\partial}{\partial w_j} Loss(\mathbf{w})$
 - The gradient tells you which direction is downhill (for a two-dimensional line, this is the slope)
- For each weight, move a small amount in the steepest downhill direction
 - The learning rate α controls how far you move
 - $w_j \leftarrow w_j - \alpha \frac{\partial}{\partial w_j} Loss(\mathbf{w})$
- Repeat until convergence at a local minimum





Linear Regression

Two Types of Gradient Descent

- Type 1: Batch gradient descent

- Process all data points at once
- Convergence is guaranteed given a small enough learning rate
- Epoch – one step that covers all the training examples

- Updating bias weight:

- $w_0 \leftarrow w_0 + \alpha \frac{1}{n} \sum_i (y_i - h_w(x_i))$

- Updating other weights:

- $w_j \leftarrow w_j + \alpha \frac{1}{n} \sum_i (y_i - h_w(x_i)) \times x_i$

- w_j : weight for feature j
- α : learning rate
- y_i : true output value for data point i
- $h_w(x_i)$: predicted value for data point i
- x_i : vector of input features for data point i
- n : number of training examples

Two Types of Gradient Descent

- Type 2: **Stochastic gradient descent (SGD)**
 - Consider a single training point (or a small **minibatch** of training points) at a time
 - Take a step after each update
 - Convergence is not guaranteed
- Updating bias weight for a single training point x_i :
 - $w_0 \leftarrow w_0 + \alpha(y_i - h_w(x_i))$
- Updating other weights for a single training point x_i :
 - $w_j \leftarrow w_j + \alpha(y_i - h_w(x_i)) \times x_i$

- w_j : weight for feature j
- α : learning rate
- y_i : true output value for data point i
- $h_w(x_i)$: predicted value for data point i
- x_i : vector of input features for data point i



Thinking it through

- We want weights that minimize empirical loss:

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} \operatorname{Loss}(h_{\mathbf{w}})$$

- Is it better to solve for weights analytically or using gradient descent? Why?
- Is it better to use batch gradient descent or stochastic gradient descent? Why?

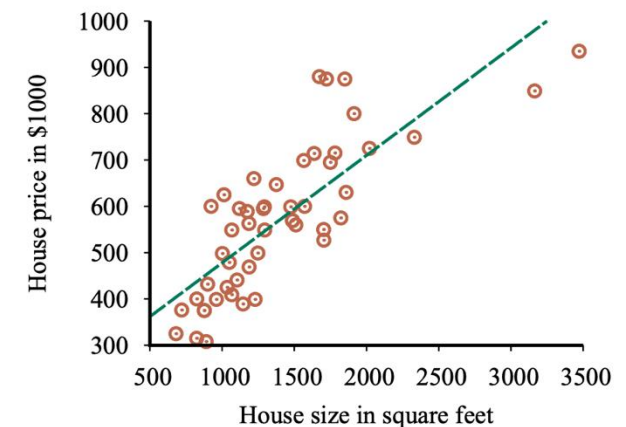
Evaluating Regression Models

- R^2 statistic (coefficient of determination)
 - measures how well a regression model fits the data
- Value of 1 – perfect relationship between true output and predicted output
- Value of 0 – no relationship between true output and predicted output

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

SS_{RES} – how much variance is due to the residual (error) in prediction

SS_{TOT} – total sum of squares; proportional to the variance of the data



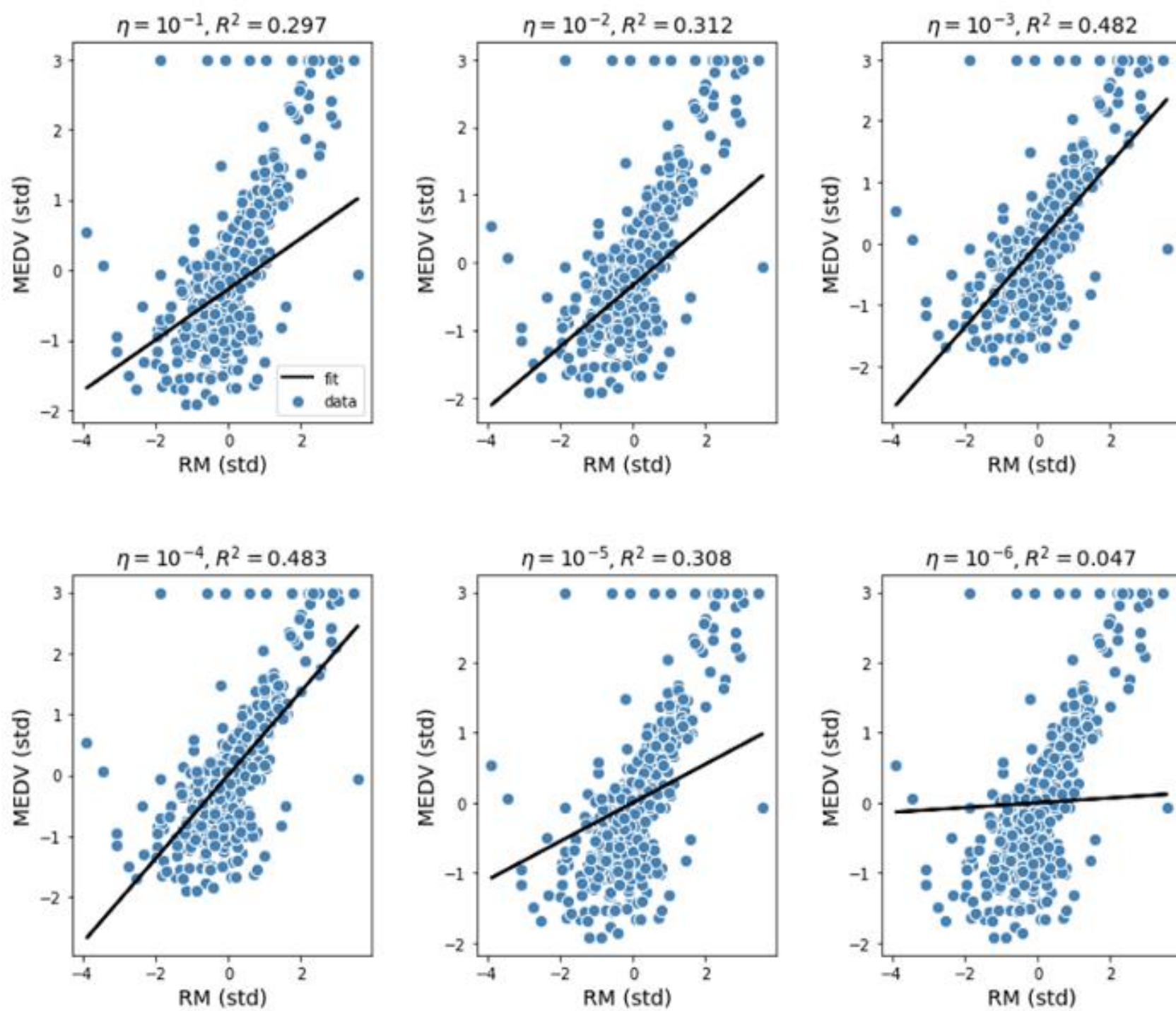


Image from
<https://benjaminobi.medium.com/what-really-is-r2-score-in-linear-regression-20cafd5b87c>

Overfitting and Regularization

- h_w : prediction function
- λ : regularization weight
- L_q : regularization function
- $\mathbf{w}$: vector of weights
- w_i : weight for feature i
- q : regularization coefficient

- There is a real danger of overfitting
 - Especially true for multivariate regression!
- Regularization
 - Penalizes complex models

Note that this L_1 and L_2 are different from the L_2 (squared loss) we saw earlier

$$Cost(h) = EmpLoss(h) + \lambda Complexity(h)$$

- For linear functions: specify this as a function of the weights

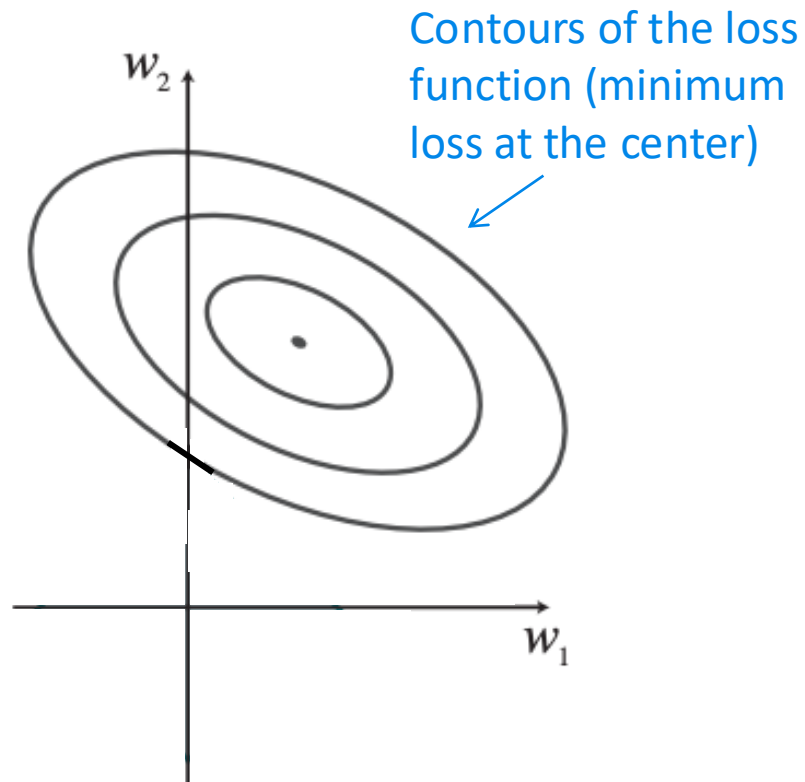
$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q$$

L_1 regularization, $q = 1$
 L_2 regularization, $q = 2$
...

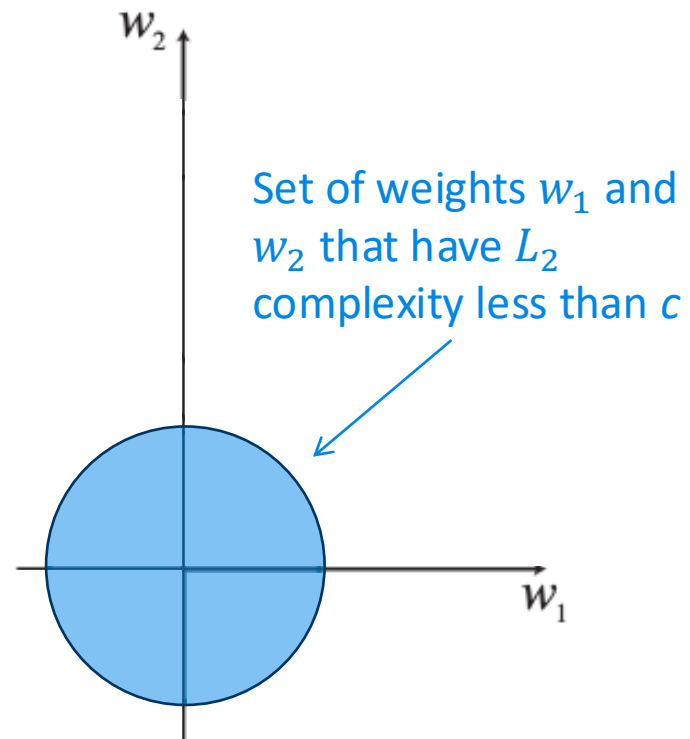
$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q \leftarrow \begin{array}{l} L_1 \text{ regularization, } q = 1 \\ L_2 \text{ regularization, } q = 2 \\ \dots \end{array}$$

Visualizing L2 Regularization

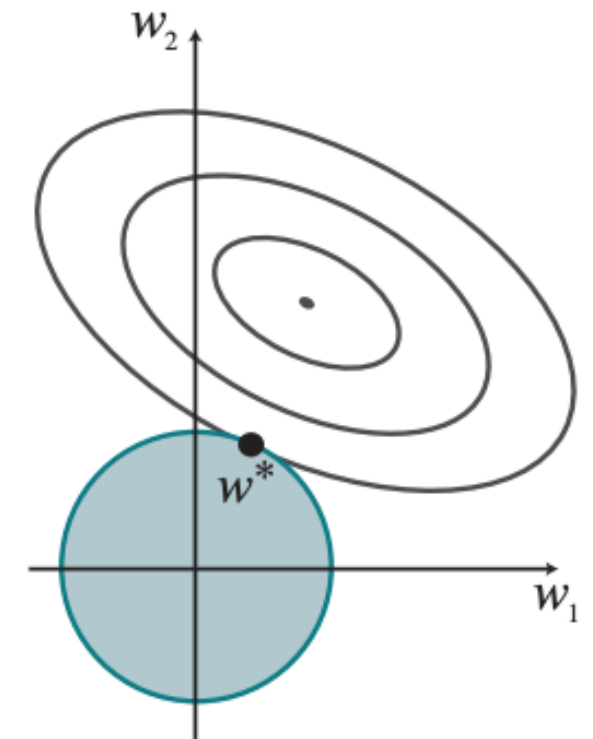
Let's visualize an example loss function with two weights:



L2 regularization will prioritize smaller weights:



Optimal weights with complexity less than c :



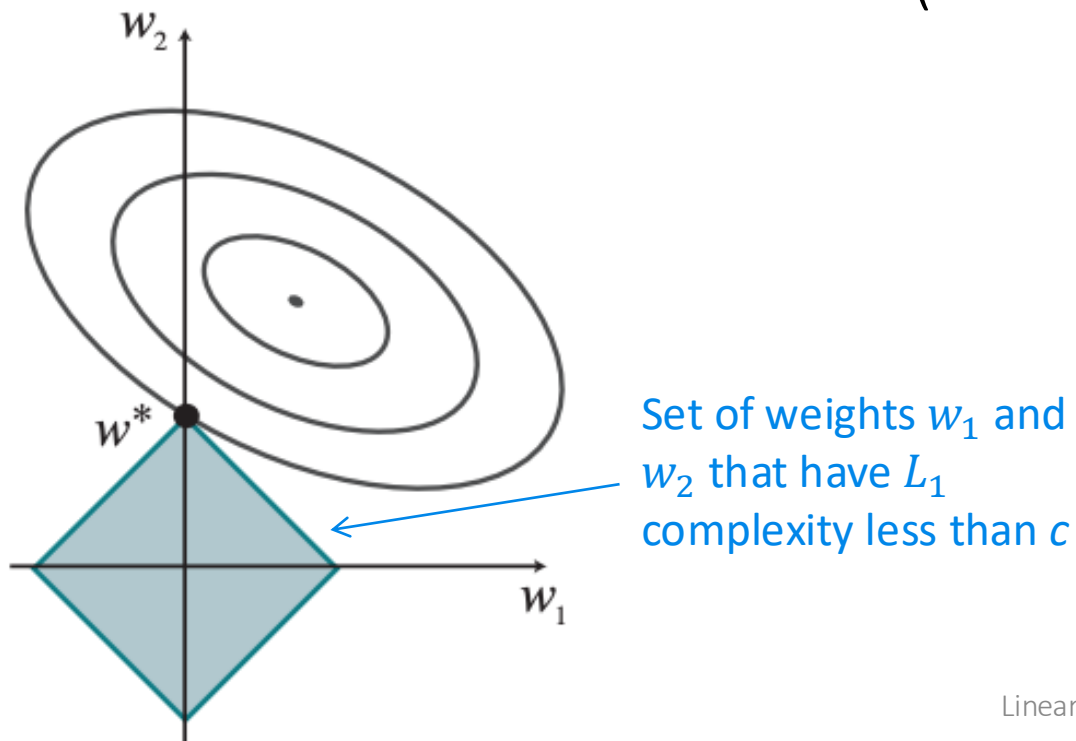
$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q$$

$\leftarrow$ L_1 regularization, $q = 1$
 L_2 regularization, $q = 2$
...

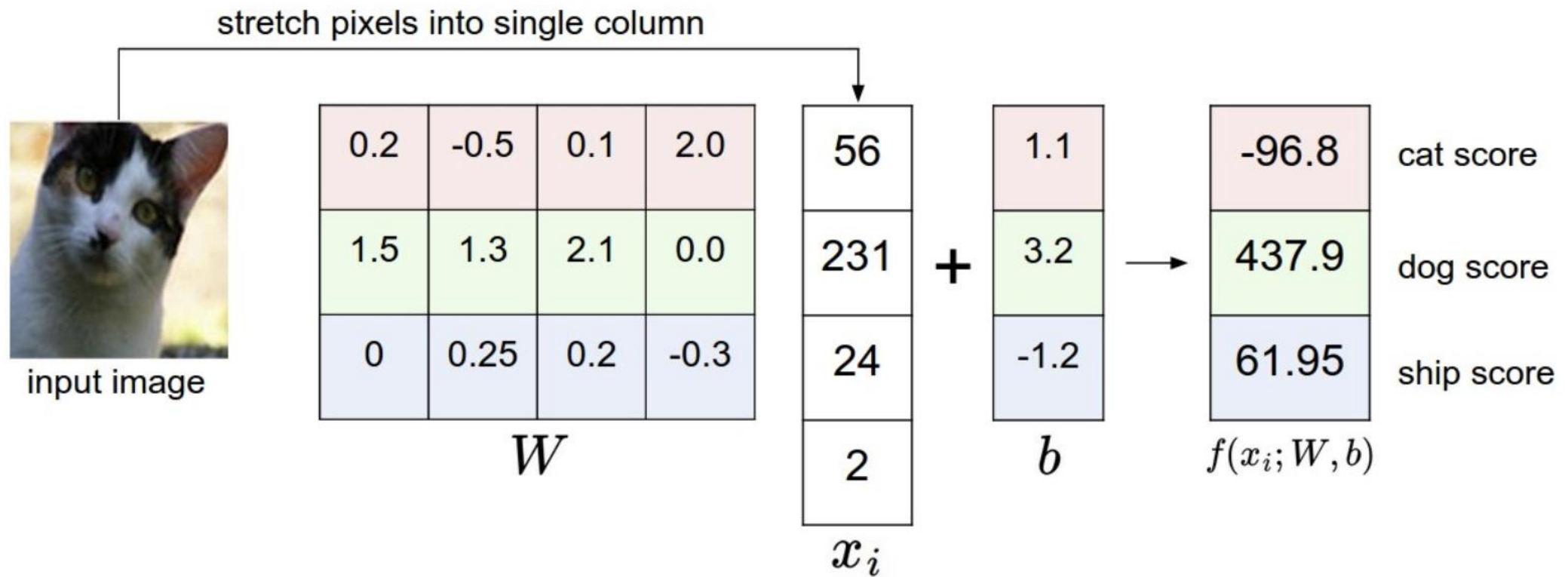
Visualizing L1 Regularization

L1 regularization has a different shape:

L1 regularization creates a **sparse model** – many weights are zero (essentially discarded)



Linear Classification



Notation

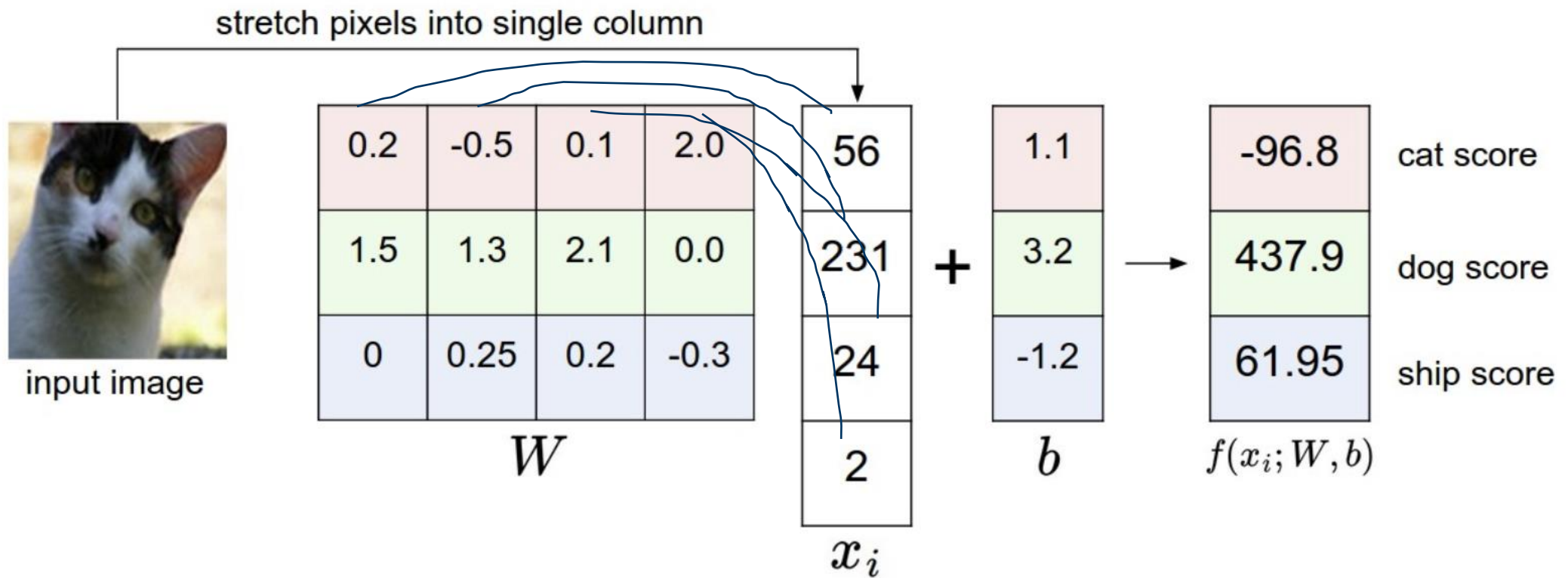
- $x_i \in \mathbb{R}^D$ training dataset of images, associated with label y_i
- $i = 1, \dots, N$ – we have N images
- $y_i \in 1, \dots, K$ – we have K classes

CIFAR-10 dataset has a training set of $N = 50k$ images, $D = 32 \times 32 \times 3 = 3072$ pixels, $K = 10$ classes (dog, cat, car, etc.)

Linear Classifier: Map image pixels to classes

- $f(x_i, W, b) = Wx_i + b$
- x_i has all of its pixels flattened out to a single column vector of shape $[D, 1]$
- Weight matrix W (size $[K \times D]$) and bias vector b are **parameters** of the function
- CIFAR-10 has each image flattened into a single $[3072 \times 1]$ column, W is $[10 \times 3072]$ and b is $[10 \times 1]$, so 3072 numbers come into the function (raw pixel values) and 10 numbers come out (class scores)

Linear Classification



Visualization

